

← Go Back

Hardware

Nano RP2040 Connect

Tutorials

Nano RP2040 Connect User Manual

Web Server AP Mode with Arduino Nano RP2040 Connect

BLE Device to Device with Nano RP2040 Connect

Nano RP2040 Connect Chromebook Setup

Nano RP2040 Datalogger with MicroPython

Using the IMU Machine Learning Core Features

Accessing IMU Data on Nano RP2040 Connect

Setting up Nano RP2040 Connect with Arduino Cloud

Reading Microphone Data on Nano RP2040 Connect

MicroPython and the IMU Machine Learning Core Features

Getting Started With the Nano RP2040 Connect and OpenMV

Control Built-in RGB LED over Wi-Fi with Nano RP2040 Connect

Voice Commands With The Arduino Speech Recognition Engine

Home / Hardware / Nano RP2040 Connect / BLE Device to Device with Nano RP2040 Connect

i This tutorial refers to a product that has reached its end-of-life status.

BLE Device to Device with Nano RP2040 Connect

Learn how to connect 2x Nano RP2040 Connect boards with each other, using Bluetooth® Low Energy.

Author · Karl Söderby · Last revision · 07/17/2024

ON THIS PAGE

Introduction

- Goals
- Hardware & Software Needed
- Programming the Board +
- Testing It Out +
- Conclusion

Introduction

In this tutorial, we will learn how to turn on the blue pixel onboard the Arduino® Nano RP2040 Connect board, from another board. For this, we will need two Bluetooth® Low Energy compatible boards, such as the Nano RP2040 Connect board, where we will use the [ArduinoBLE](#) library to make the connection.

Note: if you need help setting up your environment to use your Arduino Nano RP2040 board, please refer to [this installation guide](#).

Help

Goals

Connect two Nano RP2040
Connect boards using
Bluetooth® Low Energy.

Set up one board as a
"peripheral" and one as a
"central" device.

Set up a boolean that
switches each time a button
is pressed.

Depending on what state
the boolean is in, send a
byte to the other board
(either 0x00 or 0x01 which
represents 0 and 1).

Hardware & Software Needed

Arduino IDE ([online](#) or
[offline](#)).

[ArduinoBLE](#) library.

2x [Arduino Nano RP2040](#)
[Connect](#).

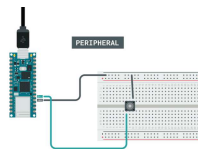
Breadboard.

Pushbutton.

Jumper wires.

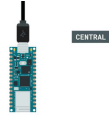
Circuit

For the peripheral device, we need
to connect a pushbutton the to
board.



The circuit for the
peripheral device.

For the central device, we do not
need any additional circuit.



The circuit for the
central device.

Programming the Board

We will now get to the
programming part of this tutorial.

- 1 First, let's make sure we have the drivers installed. If we are using the Cloud Editor, we do not need to install anything. If we are using an offline editor, we need to install it manually. This can be done by navigating to **Tools > Board > Board Manager....** Here we need to look for the **Arduino Mbed OS Nano Boards** and install it.
- 2 Now, we need to install the libraries needed. If we are using the Cloud Editor, there is no need to install anything. If we are using an offline editor, simply go to **Tools > Manage libraries...**, and search for **ArduinoBLE** and install it.

- 3 We can now take a look at some of the core functions of the sketches we will use:

`BLE.begin()` - initializes the library

```
BLE.scanForUuid("19b100
00-e8f2-537e-4f6c-
d104768a1214")
```

- scans for Bluetooth® Low Energy peripherals until the one inside parenthesis is found.

```
BLEDevice peripheral =
BLE.available()
```

checks whether peripheral has been discovered.

```
BLEDevice central =
BLE.available()
```

checks whether peripheral has been discovered.

```
while
(peripheral.connected()
)
```

- while a peripheral is connected, enter a while loop.

```
while
(central.connected())
```

- while a peripheral is connected, enter a while loop.

```
LEDCharacteristic.write
Value((byte)0x00)
```

- sends a "0" to the central.

```
LEDCharacteristic.write
Value((byte)0x01)
```

- sends a "1" to the central.

```
byte value =
LEDCharacteristic.read(
)
```

- reads incoming data.

```
LEDCharacteristic.readValue(value)
```

- stores incoming data in the `value` byte.

```
if (value == 0x00) -
```

checks if incoming data is "0"

```
if (value == 0x01) -
```

checks if incoming data is "1".

Complete Code for Peripheral Device

Upload the code below to the peripheral device.

```
1  #include <ArduinoBLE.h>
2
3  int buttonPin = 2;
4  boolean ledSwitch;
5
6  BLEService LEDService
7  // BLE LED Switch Characteristic
8  BLEByteCharacteristic
9  void setup() {
10     Serial.begin(9600);
11     pinMode(buttonPin,
12     // begin initialization
13     if (!BLE.begin()) {
14         Serial.println("BLE
15     }
16     // set advertised name
17     BLE.setLocalName("BLE
18     BLE.setAdvertisedService
19     // add the characteristic
20     LEDService.addCharacteristic
21     // add service
22     BLE.addService(LEDService
23     // start advertising
24     BLE.advertise();
25     Serial.println("BLE
26 }
27 void loop() {
28     // listen for BLE peripheral
```

Complete Code for Central Device

Upload the code below to the central device.

```
1  #include <ArduinoBLE.  
2  
3  void setup() {  
4      pinMode(LED_BUILTIN  
5      Serial.begin(9600);  
6      while (!Serial);  
7      // initialize the B  
8      BLE.begin();  
9      Serial.println("BLE  
10     // start scanning f  
11     BLE.scanForUuid("19  
12 }  
13 void loop() {  
14     // check if a perip  
15     BLEDevice periphere  
16     if (peripheral) {  
17         // discovered a p  
18         Serial.print("Fou  
19         Serial.print(peri  
20         Serial.print(" '"  
21         Serial.print(peri  
22         Serial.print("' "  
23         Serial.print(peri  
24         Serial.println();  
25         if (peripheral.lo  
26             Serial.println(  
27             return; // If  
28     }
```

Testing It Out

After we have uploaded both the **central** and **peripheral** sketches to our boards, we can test out our application.

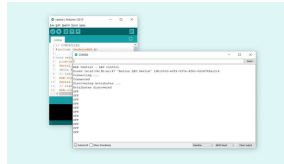
Let's start by opening the Serial Monitor of the central device.

Since we are using the

`while(!Serial);` command, the program will not initialize until we open the Serial Monitor. This is done so we don't miss any important information in the Serial Monitor.

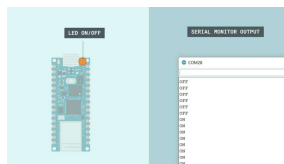
When we open it, the central

will attempt to connect to it. If all goes well, we should now start receiving the state of the button from the **peripheral device**. In this case, it is OFF, which we can see being updated continuously.



Serial Monitor printing connection status and "OFF".

Now, if we press the button on the peripheral device, we can see two things change on the **central device**. The blue LED will turn ON, and the Serial Monitor will instead print "**ON**". We can now turn ON or OFF the LED, through pushing the same button.



When the button is pressed, the RGB LED turns blue on the other device.

Troubleshoot

If the code is not working, there are some common issues we can troubleshoot:

- We have not installed the **ArduinoBLE** library.

- We have not uploaded the right sketch for the right

the other board needs the central sketch.

We have not powered both boards. Remember that both programs need to be up and running for this application to work.

Conclusion

In this tutorial, we have created a simple device-to-device application over Bluetooth®. We set up a **peripheral device** with a button, and used the built-in RGB on the **central device**. With this, we can simply turn ON and OFF the LED using Bluetooth® Low Energy, with the same button.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

